Hierarchical Metric-Semantic SLAM Methods for Architectural Mapping

Jasper Wilkes

A dissertation submitted for the degree of

BSc Computer Science

Cardiff University

February 2026

Copyright © 2026 Your Full Name

This work is licensed under the Creative Commons Attribution-ShareAlike License 4.0.
SPDX-License-Identifier: CC-BY-SA-4.0
Full text: https://creativecommons.org/licenses/by-sa/4.0/legalcode

Abstract

Simultaneous Localisation and Mapping (SLAM) is the fundamental robotics problem of estimating both the pose of a mobile robot and a map of its surroundings. This enables a robot to then make and execute tasks such as planning or navigation. Recent advancements addressing the SLAM problem use a graph-based representation of a scene that integrates metric (occupancy) and semantic (classes) estimates, along with hierarchical relationships connecting different aspects of the scene (e.g. an object belongs to a room). This representation, known as a 3D Dynamic Scene Graph (DSG), encompasses actionable information that aids in robot planning and decision-making, but also in supporting architecture-related tasks such as renovation, inventory and remote building visualisation.

This project focuses on deploying and evaluating 3 state-of-the-art DSG methods in a simulated environment: S-Graphs+ [1], Hydra [2] and Clio [3]. The best performing method was then used to process real world data and create a DSG of the Human Centred Computing and Robotics (HCCR) laboratory. Each method uses a different approach to creating the DSG: S-Graphs+ is geometry based, Hydra adopts a Spatial approach while Clio uses a Task-Driven approach.

The evaluation revealed that S-Graphs+ struggled in cluttered environments, as objects like desks/beds often prevented the method to identify wall planes, leading to incorrect classification of rooms. As well as this, S-Graphs+ is a structural method only, meaning no semantic or object level information is considered. Hydra performed well at identifying objects, with a 59% recall rate, but struggled to segment all rooms in the scene with an 30% recall rate, especially struggling with smaller rooms like bathrooms. Clio excelled at segmenting and labelling rooms correctly, however struggled at identifying objects as well as practical application.

Contents

Abstract ii

List of Figures v

Acronyms vi

Dedication vii

Acknowledgements viii

Data and Code ix

1 Introduction 1
- 1.1 Motivation and context . . . 1
- 1.2 Aims and objectives . . . 1
- 1.3 Contributions . . . 2
- 1.4 Dissertation structure . . . 2

2 Background 3
- 2.1 Context . . . 3
- 2.2 Related work . . . 4
- 2.3 Theory and Concepts . . . 4
  - 2.3.1 S-Graphs+[1] . . . 4
  - 2.3.2 Hydra[2] . . . 5
  - 2.3.3 Clio[3] . . . 7
- 2.4 Problem statement . . . 9

3 Methodology 10
- 3.1 Research design . . . 10
- 3.2 Literature and Evidence . . . 10
- 3.3 Specification . . . 10
- 3.4 Approach and Solution . . . 10
- 3.5 Experimental Setup . . . 10
- 3.6 Datasets . . . 10
- 3.7 Summary . . . 10

4 Results and Evaluation 11
- 4.1 Setup . . . 11
- 4.2 Results . . . 11

4.3 Discussion 11
4.4 Limitations 11

5 Conclusions and Future Work **12**
5.1 Conclusions 12
5.2 Future work 12

6 Reflection on Learning **13**
6.1 Critical narrative of the experience 13
6.2 Analysis and application 13
6.3 Implications for future practice 13

A Supplementary Material **14**
A.1 Additional tables 14
A.2 Further analysis 14
A.3 Derivation 14

B Additional Results **15**
B.1 Additional data 15
B.2 Further details 15

License **16**

References **17**

List of Figures

2.1 Illustration of room segmentation via graph of places dilation, reproduced from [2]. 7

Acronyms

AIB Agglomerative Information Bottleneck. i, 8

API Application Programming Interface. i

DSG 3D Dynamic Scene Graph. i, ii, 1–3

GVD Generalised Voronoi Diagram. i, 6, 8

HCCR Human Centred Computing and Robotics. i, ii, 2

IAIB Incremental AIB. i, 8

IB Information Bottleneck. i, 8

SLAM Simultaneous Localisation and Mapping. i, ii, 1, 3, 4

UK United Kingdom. i

To my family.

Acknowledgements

I thank my supervisors and colleagues for their support.

Data and Code

Data

Data supporting this dissertation are available from [repository/DOI or “available on request”]. Replace with your statement.

Code

Code produced for this dissertation is available from [repository/DOI or “available on request”]. Replace with your statement.

CHAPTER 1

1. Introduction

This chapter introduces the dissertation. It outlines the motivation and context for the work, the aims and objectives, the main contributions, and the structure of the remaining chapters. The following sections expand on each of these.

1.1 Motivation and context

SLAM is the fundamental problem in robotics of an online robot making a map of its surroundings (mapping), whilst simultaneously tracking its own position (localisation). This enables actions such as path planning. This presents a chicken and egg problem, you need the map to calculate position, yet need the position to build the map.

Classical approaches to the SLAM problem represent the environment using only metric terms such as occupancy grids, which captures the raw geometry without any structural meaning. Whilst this is enough for basic navigation, increasingly advanced robotics require a more structured understanding of the scene in order to execute higher-level functions. This is particularly apparent in architecture contexts, where tasks like inventorying or remote visualisation require an understanding of spatial structure.

Recent advancements in SLAM research use a graph-based representation of the scene that integrates metric(occupancy) and semantic (classes) estimates, along with hierarchical relationships connecting different aspects of the scene. For example, an object belongs to a room. This approach, known as a DSG, enables it to have a broader range of higher-level tasks, such as responding to natural language commands like 'pickup the red pen', or architecture-related tasks mentioned above.

Different DSG methods use different approaches to how the scene is processed. This means they differ in how the scene is constructed and labelled. This proposes 3 core research questions: Which achieves the best accuracy, which is the most practical for deployment, and which creates the richest semantic understanding? This dissertation addresses these questions through a side-by-side evaluation of 3 state-of-the-art methods, resulting in deploying the best-performing method on real-world data.

1.2 Aims and objectives

The main aim of this project was to compare how 3 state-of-the-art methods perform according to different metrics, which would allow a best-performing to be selected

to create a DSG of the HCCR laboratory. The project was broken down into 4 clear objectives:

O1: Deploy all 3 methods on example data

O2: Deploy methods on custom rosbag and store results

O3: Evaluate results and determine which is the best-performing method

O4: Use best-performing method on real-world data to generate a DSG of the HCCR laboratory

1.3 Contributions

- **Integrated RGB-D camera onto existing 3D LiDAR robotic configuration**
  - Enabled multimodal data capture
- **Created simulated office dataset using integrated robot**
  - Created a rosbag recording of scene for reproducable evaluation
- **Deployed 3 state-of-the-art DSG methods (S-Graphs+ [1], Hydra [2], Clio [3]) on the custom rosbag dataset**
  - Produced examinable results for evaluation
- **Evaluated methods based on accuracy, practicality, and richness of semantic understanding**
  - Enabled a systematic comparison and selection of the best-performing method
- **Deployed best-performing method on real-world data captured in the HCCR laboratory**
  - Generated a DSG of the scene and demonstrating the practical application of the method in a real-world context

1.4 Dissertation structure

The rest of this dissertation is organised as follows.

- **Chapter 2** (Background): context, related work, problem statement.
- **Chapter 3** (Methodology): research design, specification, system design, implementation.
- **Chapter 4** (Results and Evaluation): setup, results, evaluation, limitations.
- **Chapter 5** (Conclusions and Future Work): summary and next steps.
- **Chapter 6** (Reflection on Learning): transferable learning (omit if not required).
- **Appendix A** and **Appendix B**: supplementary material and additional results.

CHAPTER 2

2. Background

This chapter provides the background needed to understand the project: the wider context, existing solutions or related work, and a clear problem statement (and research question(s) where applicable). Replace with your own review.

2.1 Context

Autonomous robots operating in dynamic environments require rich spatial representations to perform high-level tasks. Recent advancements in this topic have led to DSGs emerging as a way to build a hierarchical representation of a world scene, a key example being Hydra [2]. These scene graphs are split up into different layers, which each represent different aspects of the scene. For example, room and object layers contain information about specific detected rooms and objects like their position and semantic label. Layers are connected in a hierarchical structure which reflects the structure of the environment. For example, building nodes contain floor nodes, which contain room nodes, which contain object nodes.

This structured approach allows a more contextual view of an environment. Giving a robot more context of its surroundings enables them to perform higher-level tasks. One application for this concept is architectural-related tasks. For example, if a robot can detect objects throughout a building, it can then be used to conduct an inventory of specific items like chairs or pens, which can further be adapted to aid in building management. Another benefit could be assisting architects. A DSG produces an abstractable view of a building, which can be used be architects to analyse relational data of the scene to aid them in further development.

The general pipeline for a DSG method starts with a metric layer, where the respective sensor data is turned into a volumetric or mesh representation, this forms the metric layer of the graph and is the minimum expected for a SLAM solution. Then, the method will analyse the metric layer to extract information about the scene. The way this is extracted depends on the method and graph structure being used. For example, S-Graphs+ [1] analyses a point cloud to segment walls using the geometry of the scene, whereas Hydra [2] uses neural networks to extract objects from RGB-D data. Each layer is constructed using the data of the layers below to eventually build a graph encompassing the whole scene.

2.2 Related work

2.3 Theory and Concepts

2.3.1 S-Graphs+[1]

S-Graphs+ jointly models a pose graph and a 3D scene graph using a 4-layer topological graph consisting of Keyframe, Wall, Room and Floor layers. Unlike the other 2 methods, its data input is from a 3D LiDAR sensor, which is pre-filtered and downsampled before processing. Robot odometry (used for SLAM) is estimated either from LiDAR measurements or robot encoders, the method is agnostic to which it comes from.

Wall Extraction
Sequential RANSAC is applied per keyframe to iteratively detect dominant wall planes. RANSAC works like a voting system to decide which points belong to the same flat surface. Only processing keyframes instead of every point of data collected is vital for computational performance. Analysing consecutive scans would likely result in the same detections, therefore processing per keyframe ensures meaningful changes are captured without processing unnecessary data. Extracted walls are transformed into global map coordinates and classified into vertical (walls) and horizontal (floor/ceiling) surfaces using their normal vector orientation. As the robot traverses the scene, the Mahalanobis distance, which is a statistical measure that accounts for uncertainty, is used to detect whether a plane is new or has been previously discovered.

Room Segmentation
This method introduces a novel strategy for segmenting rooms, it is split into 2 steps: Free-Space Clustering and Room Extraction

Free-Space Clustering:
The free-space clustering algorithm divides the free-space graphs into several clusters corresponding to rooms. The algorithm has 3 main steps: 1. Removes boundary nodes which are too close to walls/obstacles. This helps to separate rooms by creating gaps such as doorways to segregate room clusters. 2. A depth-first search is used to find isolated islands of connected nodes, which becomes clusters. Since doorways were pruned previously, these clusters can be assumed to represent rooms. 3. Removed boundary nodes are re-added to maintain full coverage of free-space. This is vital for actions like navigation.

Room Extraction:
This step combines identified clusters with detected walls within that cluster to classify room configurations. Wall planes are categorised by their dominant normal direction and the sign of the normal. This allows the system to see which walls face each other and whether they form rooms. Rooms are classified as either 2 or 4 wall rooms. A 4-wall room consists of 2 x and y direction planes near a cluster, which is

then verified by checking its width to prevent noise being classified as rooms. If 2 walls surround a cluster, it is either a partially explored room or a corridor-like structure, for which the same verifications are ran. Data association steps include using room centres to identify whether multiple candidates represent the same room. The Mahalanobis distance is also used in cases where a wall might be observed different from different viewpoints.

Floor Segmentation
This module extracts the widest wall planes within the current explored floor which can be used to determine a floor centre. It does this by using all stored walls to compute widths between all x and y direction planes. The largest of each orientation undergo a dot product check between their corresponding normal orientations which removes planes which are outside the structure. When the robot ascends or descends to a different floor, the new wall planes are stored corresponding to the floor level they are on.

Back-End
The graph is created and optimised in the back-end by minimising errors across all layers, refining both the robot's pose estimates and the structure representations simultaneously.

2.3.2 Hydra[2]

Hydra combines fast early and mid-level perception processes like local mapping with slow higher-level perception like global optimisation of the scene graph. Unlike S-Graphs+, Hydra grounds symbolic knowledge into the geometric map to create metric-semantic representation. This is useful for complex instructions which need contextual information about the environment.

The hierarchical graph contains 5 layers: Metric-Semantic 3D Mesh, Objects and Agents, Places, Rooms, Buildings. When compared to S-Graphs+, Hydra [2] represents a much more grounded and actionable representation. The nodes must contain a single parent in the layer above, they can also only connect to adjacent layers, e.g. a chair connects to a room, but not the building. Sibling nodes children are disjoint, meaning furniture in sibling rooms are completely independent of each other. Hydra uses tree decomposition to represent a complex graph as a tree structure by using bags, which contains a set of nodes from the original graph. This stops the complexity exploding with size, bounding it to the most complex local structure for example how many objects in a room.

Mesh
A 3D mesh is built using Kimera and VoxBlox by maintaining a voxel-based map within an active window around the robot. Each voxel contains information about the free space and a distribution over semantic labels via Kimera's [citation] Bayesian inference. This map is then converted to a 3D mesh using marching cubes, with a semantic label added to each mesh vertex. Operating within a specified active window is

useful for memory usage as it limits the amount of data trying to be processed at once.

Objects and Agents
This layer consists of a robot pose graph obtained using stereo or RGB-D visual inertial odometry. It also segments objects using Euclidean clustering , where vertices with the same semantic label are clustered. Overlapping objects with the same semantic class are merged if the centroid of an object is within another's bounding box.

Places
A Generalised Voronoi Diagram (GVD), represents a skeleton of free-space. It is created using a Brushfire algorithm by expanding from each mesh surface until touching, which becomes a GVD point. Whilst this captures the full shape of the free-space, it is heavy to store due to its voxel representation. So, each voxel is translated to a node with edges to adjacent voxels, groups of nearby nodes are then turned into single "place" nodes which is more memory efficient. Each node contains the position and distance to nearest obstacle from the GVD voxel, as well as the minimum distance to the nearest obstacle of all GVD voxels that the edge passes through. This information is useful for navigation tasks. As the robot traverses the scene, the algorithm is run after every update to merge newly observed data into the graph of places.

Rooms
To identify rooms, persistent homology is used on the graph of places, each room is then given a semantic label using a neural tree. For the semantic labelling, we don't have a network to identify rooms like we do for objects. Therefore, the semantics of the room must be inferred using contained objects and prior knowledge. Rooms are identified using a 2 stage process, identifying number of rooms, and assigning places to rooms via flood-fill. The graph of places is dilated to expose clusters of nodes in the environment (rooms). This works as eventually door nodes will gradually close, naturally sectioning the map into rooms.

Persistent Homology is used to compute the best dilation distance for a given graph, this allows room segmentation to be effective across different scene contexts. The Flood-Fill algorithm is then used to assign remaining nodes to the putative room. Semantic labels are assigned using a neural tree. The key insight of this process is objects which are typically associated with a room type, e.g. a fridge. Edges may also be used such as a master bedroom being next to a bathroom.

Loop Closure Detection
For each keyframe, it tries to find a past node which matches the latest one using associated Places, Objects and Appearance descriptors. Descriptors can be Hand-Crafted or Learning-based using GNNs. Hand-crafted counts object labels and uses a histogram of distance to obstacles for each place node, whereas learning-based uses bounding boxes, labels and spatial relationships for objects and distance and encodes distances between nodes to make it rotation invariant. Once a match candidate

is found, it is verified using a top-down then bottom-up strategy, this makes sure false loop closures are avoided.

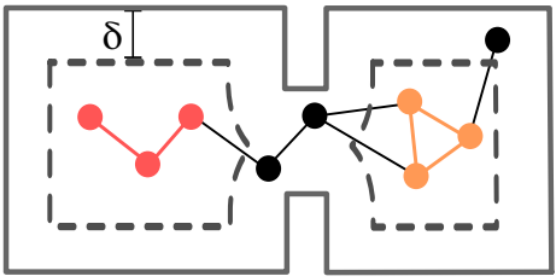

Figure 2.1: Illustration of room segmentation via graph of places dilation, reproduced from [2].

Optimisation
Odometry drift errors increase as the robot explores more of the scene, therefore the scene graph must be updated often to stay consistent. Hydra uses a sparse subgraph called a deformation graph which has robot poses, mesh control points and a sparse version of all place nodes, this representation is more efficient than optimising the entire graph every time. Each node in this graph has a local coordinate frame (position + rotation), the optimiser then finds poses which minimise deformation by respecting original measurements but allowing for some flex. You can think of it like the graph edges being rubber bands, the optimiser pulls them to satisfy loop closures while minimising how much it deforms. Once the optimiser is finished, the corrected deformation graph is interpolated to reconstruct the full scene mesh and then reconciles the graph by merging place nodes within 0.4m of each other, and objects with same semantic labels and invading bounding boxes.

2.3.3 Clio[3]

Clio is the most unique out of the 3 methods discussed in this project. The other methods build task-agnostic graphs, meaning they capture everything about the scene regardless of the robots intentions. Clio differs from this by creating a task-driven graph which only captures aspects of the scene which is relevant to the tasks the robot needs to carry out. This can be a more efficient representation as it only stores/processes things that are directly relevant, however this means the built graph cannot be used for more general operations.

Task-Aware 3D Scene Understanding
A fundamental concept for Clio is task-aware 3D scene understanding. It involves compressing a set of task-agnostic primitives X into a subset of task-relevant concepts X that are informative about the tasks Y. A naive approach would cluster segments based on embedding vectors exceeding a threshold. This would require multiple tuning thresholds such as number of segments or merge similarity, which are brittle and

scene-specific. This naturally leads to the Information Bottleneck (IB) principle, which aims to compress a given signal while preserving the mutual information between the compressed representation and another signal of interest. Clio uses this principle with the context of compressing a scene representation into clusters of relevant objects and regions for a given set of tasks, replacing thresholds with a task-driven objective.

Agglomerative Information Bottlneck

Agglomerative Information Bottleneck (AIB) is a bottom-up merging approach, meaning it starts with all individual primitives, then iteratively merges adjacent ones. Cluster merges are decided using a task-driven metric by computing the weight for each possible merge using the Jensen-Shannon divergence, then iteratively merge corresponding to the smallest weight. During each iteration, a value $\delta(k)$ is calculated as a measure for the loss of information. The algorithm is terminated once the threshold for this value is exceeded which stops losing too much information through abstraction.

Clio develops AIB further with Incremental AIB (IAIB) for real-time application. As the graph grows over time, computational complexity must be bound to maintain efficiency. IAIB handles this by clustering independently for each connected component. For example, objects in different rooms have no edges, so they can't merge anyway. This method allows $\delta(k)$ to be calculated independently for each component which maintains accuracy while bounding computational complexity.

AIB requires defining a relevance probability for each primitive, for which Clio uses CLIP. CLIP embeds both primitives and task descriptions into a shared semantic space. Then a cosine similarity score can give task-relevance per primitive. A null task with threshold a handles irrelevant primitives via pre-pruning before AIB runs. This is efficient as it reduces the graph size before the merging step which is expensive.

Front-End

Clio's front-end constructs task-agnostic object and place primitives using Khronos for 3D mesh reconstruction and primitive extraction. FastSAM and CLIP get semantic segments for each image. Each new segment is greedily associated to the candidate with the highest IoU, with no association creating a new track. Then, each track is reconstructed into a 3D primitive based on all the frames in the track, which gives a more robust semantic embedding than using 1 frame alone. Simultaneously, the background is constructed for each frame, which allows for a dense 3D model to be incrementally constructed with limited computation. To construct place primitives, it follows Hydra's approach of sparsifying a GVD into a graph of places. Then a CLIP embedding vector is used to obtain semantic features for the place.

Back-End

The back-end takes the over-segmented primitives from the front-end and applies AIB to cluster them based on their task relevance. During object detection, primitives are treated as nodes with edges defined by bounding box overlaps which ensures only physically adjacent segments are candidates for merging. When clustering places,

it applies the same principles to the places graph, grouping nodes into semantic regions based on their relevance to tasks. For example a sink and stove clustered into a kitchen region.

2.4 Problem statement

CHAPTER 3

3. Methodology

This chapter describes how the problem was addressed. Choose sections that fit your project type; omit or merge as needed. Replace with enough detail for reproduction.

3.1 Research design

3.2 Literature and Evidence

3.3 Specification

3.4 Approach and Solution

3.5 Experimental Setup

3.6 Datasets

3.7 Summary

CHAPTER 4

4. Results and Evaluation

This chapter presents the results and evaluates them. Describe how the solution was demonstrated, include test or experimental results, and critically appraise strengths and weaknesses.

4.1 Setup

4.2 Results

4.3 Discussion

4.4 Limitations

CHAPTER 5

5. Conclusions and Future Work

This chapter concludes the dissertation. Summarise the aims, main results, and what was learned; then outline future work and improvements.

5.1 Conclusions

5.2 Future work

CHAPTER 6

6. Reflection on Learning

This chapter reflects on what was learned from the project: how assumptions and decisions were re-evaluated and what would be done differently in future.

6.1 Critical narrative of the experience

6.2 Analysis and application

6.3 Implications for future practice

APPENDIX A

A. Supplementary Material

This appendix contains supplementary material: extra tables, derivations, or notation. Replace with your own content.

A.1 Additional tables

A.2 Further analysis

A.3 Derivation

APPENDIX B

B. Additional Results

This appendix contains supplementary material: extra tables, derivations, or notation. Replace with your own content.

B.1 Additional data

B.2 Further details

License

This work is licensed under the Creative Commons Attribution-ShareAlike License 4.0.
`SPDX-License-Identifier: CC-BY-SA-4.0`

> You may share and adapt the work under the terms of the Creative Commons Attribution-ShareAlike 4.0 International License: appropriate credit, link to the license, indicate changes; adaptations must be shared under the same license.
>
> Full legal code: https://creativecommons.org/licenses/by-sa/4.0/legalcode

References

[1] Hriday Bavle, Jose Luis Sanchez-Lopez, Muhammad Shaheer, Javier Civera, and Holger Voos. S-graphs+: Real-time localization and mapping leveraging hierarchical representations. *arXiv*, 2022.

[2] Nathan Hughes, Yun Chang, and Luca Carlone. Hydra: A real-time spatial perception system for 3d scene graph construction and optimization, 2022. URL https://arxiv.org/abs/2201.13360.

[3] Dominic Maggio, Yun Chang, Nathan Hughes, Matthew Trang, Dan Griffith, Carlyn Dougherty, Eric Cristofalo, Lukas Schmid, and Luca Carlone. Clio: Real-time task-driven open-set 3d scene graphs. *IEEE Robotics and Automation Letters*, 9(10):8921–8928, 2024. doi: 10.1109/LRA.2024.3451395.